Задание 2 ЕГЭ по информатике

Таблицы истинности и восстановление порядка столбцов

Этот конспект объясняет задачи, в которых дана логическая формула и несколько строк её таблицы истинности с пропусками. Нужно понять, какие переменные стоят в неизвестных столбцах. Мы последовательно разберём смысл операций, вычисление формул, ручное восстановление таблицы и проверку на Python. В конце есть упражнения с объяснёнными ответами.

Главная идея решения: порядок столбцов подходит, если при нём можно заполнить пропуски так, чтобы все строки одновременно согласовывались с формулой и выполняли условие о неповторении. По отдельности правдоподобные догадки ещё не образуют решение всей задачи.

Изучай материал небольшими частями. После каждой новой операции попробуй объяснить её своими словами и самостоятельно вычислить приведённые примеры. К программам переходи после ручных разборов: тогда ты сможешь проверить, что именно перебирает код и какие условия он учитывает.

1 Что дано и что требуется найти

Логическая формула получает значения нескольких переменных и выдаёт результат: истину или ложь. В таблицах эти результаты обозначают соответственно числами 1 и 0. Например, в выражении F = x ∧ y буквы x и y обозначают входные значения, а F — результат. Знак ∧ мы подробно разберём далее.

В обычной таблице над столбцами написаны имена переменных. В задании 2 эти подписи могут быть скрыты. Значения в некоторых клетках тоже могут отсутствовать. Требуется восстановить подписи столбцов, используя формулу и все известные значения.

Каждой переменной соответствует ровно один столбец. Если первый столбец оказался y, то он обозначает y во всех строках. Нельзя назначить ему y в первой строке и x во второй.

Допустим, установлено: первый столбец — z, второй — x, третий — w, четвёртый — y. Ответ записывают в порядке столбцов слева направо: zxwy. Перечислять номера столбцов для x, y, z, w не нужно, если условие требует именно строку букв.

Столбец 1	Столбец 2	Столбец 3	Столбец 4	F
z	x	w	y	результат

Столбец F не участвует в перестановке входных переменных. Если значений функции несколько и они различаются, каждую строку проверяют с её собственным значением F.

В подборке по ссылке встречаются формулировки о частично заполненном фрагменте с различными строками, а также отдельное указание, что приведены

все наборы с определённым значением F [1]. Это разные объёмы информации. Ниже мы разберём, когда различие влияет на решение.

2 Что означают истина и ложь

Высказывание — утверждение, о котором в рассматриваемой ситуации можно сказать: оно истинно или ложно. «Число 8 делится на 2» — истинное высказывание. «Число 8 меньше 3» — ложное. Вопрос «Сколько сейчас времени?» сам по себе не является таким высказыванием.

В этой теме используются только два логических значения: 1 означает истину, 0 означает ложь. Мы не рассматриваем отдельное значение «неизвестно». Когда в таблице есть пустая клетка, значение в ней существует и равно 0 или 1; просто нам его не сообщили.

Буква x — логическая переменная: место, куда можно подставить 0 или 1. В разных строках x может принимать разные значения. Внутри одной строки все появления x обозначают одно и то же значение. Например, нельзя в выражении $x \vee \neg x$ подставить в первое x ноль, а во второе единицу.

Слово «функция» означает, что результат однозначно определяется входами. Если в двух строках все входные значения совпали, результат F тоже обязан совпасть. Одинаковые входы с разными F не могут принадлежать одной таблице истинности одной и той же формулы.

Буквы A и B далее иногда будут обозначать целые части формулы. Например, при $A = x \vee y$ и $B = \neg z$ выражение $A \wedge B$ означает $(x \vee y) \wedge \neg z$. Для внешней операции неважно, сколько переменных внутри A: после вычисления A всё равно равно 0 или 1.

3 Отрицание

Начнём с операции, которая работает с одним значением. Отрицание меняет истину на ложь, а ложь на истину. Записывается $\neg A$ и читается «не A». Если A означает «число чётное», то $\neg A$ означает «число не является чётным».

A	¬A
0	1
1	0

Поэтому при $x = 0$ выражение $\neg x$ равно 1, а при $x = 1$ оно равно 0. Здесь 0 не «ничего»: это полноценное значение, которое отрицание превращает в 1.

Два отрицания возвращают исходное значение: $\neg\neg A = A$. Можно проверить оба случая. Из 0 первое отрицание делает 1, а второе снова 0. С единицей происходит обратная последовательность.

Скобки показывают, что именно отрицается. В $\neg(x \wedge y)$ сначала вычисляют всё $x \wedge y$, а затем меняют полученный результат. В $(\neg x) \wedge y$ сначала меняют только x. Эти выражения не равны: при $x = 0$, $y = 0$ первое равно 1, второе равно 0.

Иногда отрицание обозначают чертой над выражением. Тогда важно увидеть, над какой частью проходит черта: только над переменной или над всей группой.

4 Логическое И

Операция A ∧ B истинна, только когда оба её участника истинны. Её называют конъюнкцией. Название нужно, чтобы понимать формулировки и разборы; сам принцип — совместное выполнение двух требований.

Например, для доступа одновременно нужны пропуск и разрешение. Если отсутствует хотя бы одно, условие доступа не выполнено. Это и есть смысл И.

A	B	A ∧ B
0	0	0
0	1	0
1	0	0
1	1	1

Из результата A ∧ B = 1 следует сразу A = 1 и B = 1. Из результата A ∧ B = 0 следует только, что хотя бы один участник равен 0. Оба тоже могут быть нулями, но это не обязательно.

Для трёх частей правило сохраняется: A ∧ B ∧ C = 1, только если все три равны 1. Один ноль делает всё выражение ложным. Порядок частей при И не влияет на результат: A ∧ B = B ∧ A.

На значениях 0 и 1 результат И совпадает с обычным умножением. Это полезная проверка, но в формулах и программах лучше сначала пользоваться именно логической операцией: так яснее, какую задачу мы решаем.

5 Логическое ИЛИ

Операция A ∨ B истинна, когда истинно хотя бы одно из A и B. Её называют дизъюнкцией. Если истинны оба, результат тоже 1.

Представь условие «можно предъявить паспорт или загранпаспорт». Оно выполнено, если есть хотя бы один подходящий документ. Наличие сразу двух документов не отменяет выполнение условия.

A	B	A ∨ B
0	0	0
0	1	1
1	0	1
1	1	1

В повседневной речи «или» иногда означает «одно из двух, но не оба». В обычной логической дизъюнкции это ограничение отсутствует. Операцию с таким ограничением разберём отдельно.

Из A ∨ B = 0 следует, что A = 0 и B = 0. Из A ∨ B = 1 следует только, что хотя бы один участник равен 1. Делать вывод «оба равны 1» нельзя.

Для нескольких частей: A ∨ B ∨ C = 0, только если все три равны 0. Одна единица делает всё выражение истинным. Перестановка частей не меняет результата.

Обычное сложение не совпадает с логическим ИЛИ во всех случаях. При 1 и 1 сумма равна 2, а результат ИЛИ равен 1. Поэтому заменять ∨ знаком + без дополнительного объяснения опасно.

6 Импликация и смысл условия

Запись A → B читают «если A, то B». Операция называется импликацией. Часть A стоит перед стрелкой и задаёт условие. Часть B стоит после стрелки и задаёт требование при выполнении условия.

Чтобы понять таблицу, представь правило: «Если участник набрал проходной балл, его допускают к следующему этапу». Обозначим через A факт набора проходного балла, через B — допуск. Нарушение правила обнаружено, если балл набран, а допуска нет. Это случай A = 1, B = 0.

Если A = 1 и B = 1, правило выполнено. Если A = 0, правило само по себе ничего не требует о B. Участника могли допустить по другому основанию или не допустить. Ни один из этих случаев не является нарушением записанного условия.

A	B	A → B	Почему
0	0	1	Условие A не выполнено; запрещённого случая нет
0	1	1	Условие A не выполнено; B допустимо
1	0	0	Выполнено A, но не выполнено требование B
1	1	1	При выполненном A выполнено и B

Так устроена именно логическая импликация. Её значение не утверждает, что A является причиной B. Из истинности A → B также не следует, что само A истинно.

Удобно запомнить один запрещённый случай: 1 → 0. Остальные три дают 1. Тогда всю таблицу можно восстановить, а не заучивать отдельно четыре строки.

Выведем замену стрелки. Чтобы не попасть в запрещённый случай, достаточно, чтобы A было ложным или B было истинным. Поэтому A → B = ¬A ∨ B. Проверка: при A = 1, B = 0 получаем 0 ∨ 0 = 0; в остальных случаях хотя бы одна часть равна 1.

Обратное направление обычно отличается: A → B и B → A — разные выражения. При A = 0, B = 1 первое равно 1, второе 0. Направление стрелки всегда нужно сохранять.

Полезны четыре частных случая. Если A = 0, то A → B всегда равно 1. Если B = 1, результат тоже всегда 1. Если A = 1, результат равен B. Если B = 0, результат равен ¬A. Последние два правила легко получить из замены ¬A ∨ B.

7 Эквивалентность и исключающее ИЛИ

Операция A ≡ B истинна, когда значения A и B одинаковы. Её называют эквивалентностью, или равнозначностью. Оба значения могут быть как единицами, так и нулями.

A	B	A ≡ B
0	0	1
0	1	0
1	0	0
1	1	1

Следовательно, результат 1 означает «одинаковы», а результат 0 означает «различаются». Из A ≡ B = 1 нельзя сделать вывод, что A = B = 1: случай 0 и 0 тоже подходит.

Равнозначность можно раскрыть так: (A ∧ B) ∨ (¬A ∧ ¬B). Первая часть разрешает две единицы, вторая — два нуля. Ещё одна запись: (A → B) ∧ (B → A). Две стрелки запрещают оба случая несовпадения: 1,0 и 0,1.

Иногда используется исключающее ИЛИ, обозначаемое ⊕. Для двух значений оно истинно тогда, когда значения различаются: 0 ⊕ 1 = 1 и 1 ⊕ 0 = 1; одинаковые значения дают 0. Значит, A ⊕ B = ¬(A ≡ B).

Если в задаче встретится незнакомый знак, сначала найди его определение в условии. Например, A ↑ B иногда означает ¬(A ∧ B), а A ↓ B — ¬(A ∨ B). Это отрицания уже известных операций. Не нужно приписывать незнакомому обозначению смысл по внешнему виду.

8 Как читать составную формулу

Формула строится из небольших частей. Сначала нужно понять, к каким частям относится каждая операция, затем вычислять результаты от внутренних частей к внешним.

В учебной записи обычно используют порядок: скобки, отрицание, И, ИЛИ, импликация, эквивалентность. Если условие специально задаёт другой порядок, следуют условию. Для дополнительных знаков, например ⊕, не стоит угадывать приоритет: используй скобки и определение из задачи.

Например, ¬x ∨ y ∧ z означает (¬x) ∨ (y ∧ z). Здесь И выполняется раньше ИЛИ. Выражение (¬x ∨ y) ∧ z устроено иначе. При x = 0, y = 0, z = 0 первое равно 1, а второе равно 0.

Разберём формулу F = ((¬x ∨ y) → z) ≡ (x ∧ w). Последняя, внешняя операция здесь — эквивалентность. Её левый участник — вся импликация, правый — всё x ∧ w. Нельзя сравнивать z и x, пропустив окружающие скобки.

Пусть x = 1, y = 0, z = 0, w = 1. Тогда последовательно получаем: ¬x = 0; ¬x ∨ y = 0; 0 → z = 0 → 0 = 1; x ∧ w = 1. В конце 1 ≡ 1 = 1. Значит, F = 1.

Теперь поменяем только w на 0. Левая часть останется равной 1, правая станет 0. Получим 1 ≡ 0 = 0. Даже если переменная встретилась один раз, её изменение может изменить всю функцию.

Импликацию нельзя произвольно перегруппировывать. Например, при A = B = C = 0 выражение (A → B) → C равно 1 → 0 = 0, а A → (B → C) равно 0 → 1 = 1. При нескольких стрелках особенно внимательно сохраняй скобки.

9 Как построить полную таблицу истинности

Полная таблица содержит все возможные наборы входных значений. Для одной переменной есть два набора: 0 и 1. Для двух — четыре: 00, 01, 10, 11. Для каждой уже имеющейся комбинации добавление новой переменной даёт два продолжения: с 0 и с 1.

Поэтому для n переменных число строк равно 2^n. Степень означает, что двойка умножается сама на себя n раз. Для трёх переменных получаем 2 · 2 · 2 = 8 строк, для четырёх — 16. Число появлений букв в формуле не влияет на число строк: учитывается число различных переменных.

Для удобства наборы можно выписать в порядке двоичного счёта. При трёх переменных последняя меняется в каждой строке, средняя — через две строки, первая — через четыре. Важен полный перебор без пропусков, а не выбранный порядок строк.

Построим таблицу для F = (x ∨ y) ∧ (x → z). Сначала вычислим два промежуточных столбца: A = x ∨ y и B = x → z. После этого F = A ∧ B.

x	y	z	A = x ∨ y	B = x → z	F = A ∧ B
0	0	0	0	1	0
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	1	0	0
1	0	1	1	1	1
1	1	0	1	0	0
1	1	1	1	1	1

Обязательно подписывай порядок переменных рядом с выписанными наборами. Запись 101 сама по себе не говорит, какая переменная равна нулю. При порядке xyz это y = 0, а при порядке yzx это z = 0.

Если нужны только строки с F = 1, можно оставить 010, 011, 101, 111 в порядке xyz. Но это строки, отобранные по значению функции. Их количество не обязано равняться числу строк, показанных в условии: условие может содержать только часть из них.

10 Как рассуждать от результата к переменным

При обычном вычислении известны входы, и мы ищем F. В задании с фрагментом часто выгодно двигаться в обратную сторону: известно F, и мы выводим ограничения на входы.

Известный результат	Что обязательно следует
¬A = 0	A = 1
¬A = 1	A = 0
A ∧ B = 1	A = 1 и B = 1
A ∧ B = 0	A = 0 или B = 0, возможно оба
A ∨ B = 0	A = 0 и B = 0
A ∨ B = 1	A = 1 или B = 1, возможно оба
A → B = 0	A = 1 и B = 0
A → B = 1	Допустимы 00, 01, 11; запрещено 10
A ≡ B = 1	A и B одинаковы
A ≡ B = 0	A и B различаются

Обрати внимание на «и» и «или» в правой колонке. Если написано «или», возникают несколько допустимых случаев. Нельзя выбрать один без дополнительного основания. Например, из A ∧ B = 0 нельзя сразу заключить A = 0: возможны A = 1, B = 0.

Вернёмся к F = (x ∨ y) ∧ (x → z) и потребуем F = 1. Поскольку снаружи стоит И, обе части должны быть истинны. Теперь разделим рассуждение по x.

Если x = 0, первая часть превращается в y, а вторая автоматически равна 1. Поэтому y = 1, а z может быть любым. Получаем 010 и 011.

Если x = 1, первая часть автоматически равна 1, а вторая равна z. Поэтому z = 1, а y может быть любым. Получаем 101 и 111. Мы нашли те же четыре строки, не вычисляя все восемь полностью.

Здесь «z может быть любым» означает свободу внутри рассмотренного случая. После учёта других клеток таблицы или неповторения строк свобода может исчезнуть. Это не окончательное разрешение поставить любое значение независимо от остальных условий.

При F = 0 у этой формулы хотя бы одна из двух частей равна 0. Первая часть ложна при x = y = 0; тогда z свободно. Вторая часть ложна при x = 1, z = 0; тогда y свободно. Получаем 000, 001, 100, 110. В данном примере случаи не пересекаются, поскольку требуют разных x.

11 Полезные преобразования и почему они работают

Преобразование формулы допустимо, если результат сохраняется при любом наборе её переменных. Такие формулы называют равносильными. Совпадение на

одной проверенной строке ещё не доказывает равносильность: нужно обосновать правило в целом или проверить все наборы.

Мы уже получили A → B = ¬A ∨ B. Теперь отрицание импликации: ¬(A → B) = A ∧ ¬B. Ведь отрицание истинно ровно там, где сама стрелка ложна, а это единственный случай A = 1, B = 0.

Отрицание ИЛИ: ¬(A ∨ B) = ¬A ∧ ¬B. Чтобы «хотя бы одно выполнено» было ложью, оба требования должны не выполняться. Отрицание И: ¬(A ∧ B) = ¬A ∨ ¬B. Чтобы совместное выполнение не состоялось, достаточно отсутствия хотя бы одного требования. Эти два правила называют законами де Моргана.

Они объясняют, почему при переносе отрицания внутрь скобок нужно и изменить каждую часть, и поменять И на ИЛИ или наоборот. Формула ¬(A ∧ B) не равна ¬A ∧ ¬B. При A = 1, B = 0 первая истинна, вторая ложна.

Ещё несколько правил следуют прямо из смысла операций: A ∧ 0 = 0; A ∧ 1 = A; A ∨ 0 = A; A ∨ 1 = 1. Повторение требования ничего не меняет: A ∧ A = A и A ∨ A = A. Хотя бы одно из A и ¬A обязательно истинно, поэтому A ∨ ¬A = 1. Одновременно они истинными быть не могут, поэтому A ∧ ¬A = 0.

Распределение И относительно ИЛИ: A ∧ (B ∨ C) = (A ∧ B) ∨ (A ∧ C). Слева требуется A и хотя бы одно из B, C. Справа перечислены два способа выполнить это: A вместе с B или A вместе с C. Возможность одновременного выполнения обоих способов результат не меняет.

Полезно также A ∨ (A ∧ B) = A. Если A истинно, всё ИЛИ истинно. Если A ложно, ложны обе части. Это правило называют поглощением. Знать название необязательно для решения, но само сокращение иногда помогает.

Импликация A → B равносильна ¬B → ¬A. Раскрывая вторую стрелку, получаем B ∨ ¬A, то есть ту же формулу с переставленными частями ИЛИ. Это преобразование называют контрапозицией. Оно отличается от ошибочной замены на B → A.

Не нужно усложнять простую задачу длинными преобразованиями. Преобразуй выражение, когда новое представление даёт понятное ограничение: фиксированное значение, равенство двух переменных или запрещённую пару.

12 Что на самом деле означает фрагмент с пропусками

В следующих таблицах символ * будет обозначать пустую клетку условия. Это обозначение для удобства чтения, а не умножение и не третье логическое значение.

Каждый пропуск допускает 0 или 1, пока другие условия не исключат один из вариантов. Разные звёздочки не обязаны заполняться одинаково. Например, строка 0, *, 1 совместима с 0, 0, 1 и с 0, 1, 1.

Строки фрагмента могут быть расположены в любом порядке. Если программа напечатала подходящие наборы в порядке 000, 001, 010, это ничего не говорит о порядке строк в задании. Сопоставлять первую напечатанную строку с первой строкой условия без проверки нельзя.

Фраза «все строки различны» относится к полностью восстановленным наборам. Две строки вида *, *, 0 внешне одинаковы, но могут после заполнения стать 1, 0, 0 и 1, 1, 0. Они допустимы. А разные на вид шаблоны могут вынужденно восстановиться в один и тот же набор; тогда использовать их вместе нельзя.

Слово «фрагмент» само по себе не обещает, что перечислены все строки с данным F. Например, если F = 1 на четырёх наборах, в условии могут показать только три. Недостающая строка остаётся правильной строкой полной таблицы, но не обязана появляться во фрагменте.

Если явно сказано «приведены все наборы, при которых F = 0», это дополнительное условие полноты. Тогда нельзя оставить подходящий нулевой набор неиспользованным. При этом порядок показанных строк всё равно может быть произвольным.

Это различие особенно важно при подсчёте нулей и единиц. Если среди всех подходящих строк переменная x равна 0 только один раз, то столбец с двумя известными нулями в разных строках не может быть x. Но если x равна 1 в трёх из четырёх подходящих строк, фрагмент из двух строк не обязан содержать именно какое-то заранее заданное число единиц.

Условие о различных строках позволяет рассуждать о числе доступных наборов. Если при некотором порядке столбцов первая и вторая строки могут восстановиться только в один и тот же единственный набор, этот порядок не подходит. Проверки «для каждой строки найдётся хоть один вариант» недостаточно: варианты должны выбираться одновременно и без повторений.

13 Универсальная последовательность ручного решения

1. Перепиши формулу, сохранив все скобки, отрицания и направления стрелок. Посчитай различные переменные. Отдельно запиши значения F в каждой показанной строке.

2. Пронумеруй неизвестные столбцы: C1, C2, C3 и при необходимости C4. Эти обозначения означают позиции, а не новые переменные формулы.

3. Найди внешнюю операцию. Если F = 0 и снаружи ИЛИ, потребуй нулей у всех частей. Если F = 1 и снаружи И, потребуй единиц у всех частей. Если внешняя операция другая, используй её таблицу.

4. Получи подходящие наборы в фиксированном порядке переменных. Можно вывести их по случаям или построить полную таблицу. Для четырёх переменных полный перебор содержит всего 16 наборов.

5. Ищи наиболее сильные признаки. Переменная может быть постоянной во всех подходящих наборах. Две переменные могут всегда совпадать или различаться. Набор с единственной единицей может однозначно определить её столбец. Используй только признаки, которые действительно следуют из формулы.

6. Сопоставляй сразу несколько строк. Учитывай уже известные цифры, общую для всех строк перестановку столбцов и неповторение. Если есть несколько возможностей, выпиши их явно и исключай противоречивые.

7. После нахождения порядка восстанови хотя бы один допустимый вариант всех строк. Подставь каждую строку обратно в исходную формулу, проверь F и различие наборов. Проверка исходной формулы защищает от ошибки при преобразованиях.

8. Запиши буквы в порядке столбцов слева направо. Не переставляй их обратно в привычный порядок x, y, z, w.

Метод универсален по логике проверки. Короткий ручной ход зависит от конкретной формулы. Если удобного признака не видно, полный перебор всех наборов и порядков столбцов остаётся конечным и надёжным способом.

14 Разбор с тремя переменными и неповторяющимися строками

Рассмотрим наш учебный пример. Формула F = (x ∨ y) ∧ (x → z). Даны три различные строки с F = 1. Нужно установить порядок переменных.

Строка	C1	C2	C3	F
1	*	*	0	1
2	*	*	0	1
3	0	1	1	1

Мы уже вывели все подходящие наборы в порядке xyz: 010, 011, 101, 111. Сопоставим, сколько раз каждая переменная равна 0 среди этих четырёх различных наборов.

Переменная	Наборы, где она равна 0	Количество
x	010 и 011	2
y	101	1
z	010	1

В C3 стоят два известных нуля в двух разных строках. Значит, переменная этого столбца должна быть равна 0 как минимум в двух различных подходящих наборах. Только x имеет такое свойство. Получаем C3 = x.

Почему нельзя выбрать y? Тогда обе первые строки были бы вынуждены стать набором 101 в порядке xyz: другого подходящего набора с y = 0 нет. Строки повторились бы. По той же причине нельзя выбрать z: для z = 0 есть только 010.

Теперь третья строка сообщает x = 1. В нашей формуле при x = 1 первая часть истинна, а стрелка x → z требует z = 1. Среди оставшихся столбцов третьей строки C1 равен 0, C2 равен 1. Поэтому C2 = z, а C1 = y.

Ответ: yzx. Проверим восстановление. В первых двух строках x = 0, поэтому для F = 1 требуется y = 1, а z может быть 0 или 1. Чтобы строки различались, значения z в них должны быть разными.

Строка	y	z	x	F
1	1	0	0	1
2	1	1	0	1
3	0	1	1	1

Можно поменять значения z в первых двух строках местами. Получится другое заполнение пропусков с тем же порядком столбцов. Это нормальная ситуация: вопрос задачи относится к порядку переменных, а не обязательно к единственному заполнению всех клеток.

Также обрати внимание: набор 111 из полной таблицы здесь не использован. Это допустимо, потому что условие говорит о трёх строках фрагмента, а не обо всех четырёх строках с F = 1.

15 Разбор с четырьмя переменными

Возьмём формулу F = (x → y) ∧ (y → z) ∧ (z → w). В каждой показанной строке F = 1. Строки различны.

Строка	C1	C2	C3	C4	F
1	0	*	1	*	1
2	1	0	*	0	1
3	*	0	1	1	1

Снаружи стоит И, поэтому истинны все три стрелки. Каждая стрелка запрещает переход от 1 слева к 0 справа. Значит, в последовательности значений x, y, z, w после первой единицы уже не может появиться ноль.

Итак, подходят только 0000, 0001, 0011, 0111, 1111. Это все варианты: либо единиц нет, либо первая единица стоит в одной из четырёх позиций. Речь сейчас идёт о порядке переменных xyzw, а не о скрытом порядке столбцов.

Посмотрим на известные цифры. Если в одной строке значение некоторого столбца равно 0, а другого — 1, то переменная с нулём должна стоять раньше переменной с единицей в цепочке x, y, z, w. Иначе единица оказалась бы перед нулём, что запрещено.

Первая строка даёт: переменная C1 раньше переменной C3. Вторая строка даёт: переменные C2 и C4 раньше переменной C1. Третья строка даёт: переменная C2 раньше переменной C4.

Объединяя эти требования, получаем единственный порядок позиций вдоль цепочки: C2, затем C4, затем C1, затем C3. Следовательно, C2 = x, C4 = y, C1 = z, C3 = w.

Но в ответе нужен порядок столбцов таблицы: C1, C2, C3, C4. Поэтому ответ zxwy. Восстановим строки в этом порядке.

Строка	z	x	w	y	F
1	0	0	1	0	1
2	1	0	1	0	1
3	1	0	1	1	1

В привычном порядке xyzw это 0001, 0011, 0111. Все три набора удовлетворяют цепочке импликаций и различаются. Все заданные цифры сохранены.

Этот пример показывает, что можно решать и через отношения между переменными. Не всегда нужно последовательно угадывать каждый пропуск. Но вывод о порядке нулей и единиц справедлив здесь благодаря конкретной цепочке импликаций, а не для любой таблицы истинности.

16 Разбор с разными значениями функции

У разных строк F может отличаться. Возьмём знакомую формулу $F = (x \vee y) \wedge (x \to z)$, но новый фрагмент. Строки различны.

Строка	C1	C2	C3	F
1	*	0	0	0
2	0	*	1	1
3	*	1	1	0

Для F = 0 наборы в порядке xyz: 000, 001, 100, 110. Для F = 1: 010, 011, 101, 111. Строки первой и третьей позиции нужно искать в первом списке, второй позиции — во втором.

Третья строка содержит две известные единицы и имеет F = 0. Из нулевых наборов только 110 содержит хотя бы две единицы. Значит, третья строка — именно x = 1, y = 1, z = 0. Нулю остаётся место только в C1, поэтому C1 = z.

Теперь вторая строка сообщает z = 0 и F = 1. Среди единичных наборов только 010 имеет z = 0. В нём x = 0, y = 1. Известная единица в C3 означает y, а C2 означает x. Ответ zxy.

Первая строка при этом задаёт x = y = 0. Тогда первая часть x ∨ y ложна, и вся функция равна 0 при любом z. В её пропуск можно поставить 0 или 1. Оба заполнения допустимы, порядок столбцов от этого не меняется.

Этот разбор ещё раз показывает, зачем отделять F от входных столбцов. Если отфильтровать только строки с F = 0, вторая строка условия исчезнет из рассмотрения и решение окажется неверным.

17 Почему иногда остаётся несколько вариантов

Перестановка — расположение одних и тех же различных элементов в разном порядке. Например, для x, y, z есть шесть порядков: xyz, xzy, yxz, yzx, zxy, zyx. Для первого места есть три выбора, для второго — два оставшихся, для третьего — один. Получается $3 \cdot 2 \cdot 1 = 6$.

Для четырёх переменных порядков $4 \cdot 3 \cdot 2 \cdot 1 = 24$. Такое произведение сокращённо обозначают 4! и называют факториалом четырёх. Не путай: $2^4 = 16$ — число наборов значений, а $4! = 24$ — число порядков названий столбцов. Это разные объекты перебора.

Если несколько порядков удовлетворяют всем выписанным условиям, нельзя выбирать первый наугад. Сначала нужно проверить копирование таблицы и формулы, значения F, неповторение и слова «все наборы». Возможно, какая-то информация потеряна.

Но иногда данные действительно не различают две переменные. Например, F = x ∧ y симметрична относительно x и y: обмен этих букв не меняет результата ни в одной строке. Если нет дополнительных сведений о подписях, по значениям одной этой функции невозможно отличить x от y.

Отсюда полезный диагностический принцип: полная таблица помогает проверить формулу, но не обещает, что любая придуманная задача про неизвестные подписи имеет единственный ответ. В корректно составленном упражнении на единственный порядок должно быть достаточно различающей информации.

Не следует путать несколько порядков столбцов и несколько заполнений пропусков. В разборе с ответом yzx было два заполнения первых строк, но один порядок букв. Для ответа этого достаточно.

18 Как перевести логические операции на Python

Python умеет работать с логическими значениями False и True. Они означают ложь и истину. В наших вычислениях входы будут целыми числами 0 и 1. Для логических проверок 0 считается ложью, 1 — истиной. Другие числа сюда подставлять не будем.

В математической записи	На Python	Смысл
¬A	not A	Отрицание
A ∧ B	A and B	И
A ∨ B	A or B	ИЛИ
A → B	(not A) or B	Запрет случая 1,0
A ≡ B	A == B	Совпадение значений
A ⊕ B	A != B	Различие значений

Знак = в программе присваивает значение имени. Например, x = 1 означает «теперь x хранит 1». Два знака == сравнивают значения. Запись x == y не меняет ни x, ни y, а выдаёт результат сравнения.

Слова and и or в Python могут возвращать один из своих участников, а не обязательно именно True или False. При входах 0, 1 и логических результатах это не нарушает вычислений. Если хочется явно получить число 0 или 1, используй int(F): преобразование int(True) даёт 1, int(False) даёт 0 [2].

Сначала переводи формулу по частям. Для F = ((¬x ∨ y) → z) ≡ (x ∧ w) удобно записать:

```
a = (not x) or y
b = (not a) or z
c = x and w
F = (b == c)
```

В первой строке вычисляется левая часть стрелки. Во второй — сама стрелка a → z. В третьей — правая часть эквивалентности. В четвёртой сравниваются уже готовые значения. Такой перевод легко сверить с ручным вычислением.

Скобки в Python особенно важны. Приоритеты сравнений и логических слов не повторяют в точности привычную математическую запись. Пиши (x == y) or z, если сравнение должно быть отдельной частью. Пиши x == (y or z), если x сравнивается со всем результатом ИЛИ.

Не используй ~ вместо not: для целых чисел это другая операция, и ~0 даёт −1, а ~1 даёт −2. Знаки & и | тоже имеют другие правила и приоритеты. Для начала and, or, not делают смысл программы наиболее прозрачным.

Иногда импликацию сокращают до A <= B. На значениях 0 и 1 это верно: обычное сравнение «меньше или равно» ложно только для 1 <= 0. Но такое сокращение работает благодаря выбранным бинарным значениям и требует аккуратных скобок.

Например, математическое (x → y) ≡ z переводится как (x <= y) == z. Запись x <= y == z имеет другой смысл: Python понимает её как совместную проверку x <= y и y == z. Это называется цепочкой сравнений [2].

При x = y = z = 0 правильная запись даёт True == 0, то есть False. Запись без скобок даёт две истинные проверки и результат True. Ошибка меняет таблицу. На первом этапе лучше раскрывать стрелку через not и or.

19 Первая программа для таблицы истинности

Начнём с обычных циклов. Цикл for повторяет действия для нескольких значений. Запись for x in (0, 1): означает: сначала выполнить блок с x = 0, затем с x = 1. Двоеточие начинает блок, а отступы показывают, какие строки к нему относятся.

Если внутрь первого цикла поместить второй, второй полностью выполнится для каждого значения первого. Поэтому два таких цикла дадут 00, 01, 10, 11. Три вложенных цикла дадут все восемь троек.

Для нашей формулы F = (x ∨ y) ∧ (x → z) получаем полноценную программу:

```
print('x y z F')

for x in (0, 1):
    for y in (0, 1):
        for z in (0, 1):
            F = (x or y) and ((not x) or z)
            print(x, y, z, int(F))
```

Команда print выводит переданные значения. Последняя строка находится внутри всех трёх циклов, поэтому запускается для каждого набора. Если случайно сдвинуть её влево, она может выполняться реже, и часть строк исчезнет.

Для вывода только строк с F = 1 добавим условие if. Оно запускает свой блок только при истинности указанной проверки:

```
for x in (0, 1):
    for y in (0, 1):
        for z in (0, 1):
            F = (x or y) and ((not x) or z)
            if F == 1:
                print(x, y, z)
```

Результат в порядке xyz:

```
0 1 0
0 1 1
1 0 1
1 1 1
```

Для нулевых строк нужно написать if F == 0:. Сокращение if not F: означает то же самое для наших бинарных значений. Для единичных строк можно написать if F:, но явное сравнение на первом этапе легче читать.

Если переменных четыре, добавь четвёртый вложенный цикл, обнови формулу и вывод. Важно менять всё согласованно: нельзя вычислять формулу с w, не присвоив w значение в текущем наборе.

20 Короткая запись полного перебора

В стандартной библиотеке Python есть модуль itertools. Модуль — набор готовых инструментов. Из него нам понадобятся product для наборов значений и permutations для порядков элементов.

Запись from itertools import product делает имя product доступным в программе. Вызов product((0, 1), repeat=3) последовательно создаёт все тройки из нулей и единиц. Аргумент repeat=3 указывает число позиций. По действию это соответствует трём вложенным циклам [3].

```
from itertools import product

for x, y, z in product((0, 1), repeat=3):
    F = (x or y) and ((not x) or z)
    if F == 1:
        print(x, y, z)
```

Каждая полученная тройка раздаётся трём именам x, y, z по порядку. Это называется распаковкой. Например, из тройки (0, 1, 1) получаются x = 0, y = 1, z = 1. Число имён должно совпадать с длиной полученного набора.

Сами наборы product выдаёт как кортежи. Кортеж — упорядоченная группа значений, например (0, 1, 1). Нам важно, что первое, второе и третье места различаются, даже если какие-то значения одинаковы.

Инструмент permutations('xyz') выдаёт все шесть порядков букв. Каждый результат — кортеж вроде ('z', 'x', 'y'). Повторов букв в таком порядке нет, потому что исходные буквы различны. product создаёт комбинации значений по позициям, permutations переставляет имеющиеся элементы [3].

Для ручного сопоставления часто достаточно программы, которая печатает подходящие наборы. Полностью автоматическое восстановление подписей полезно как второй способ проверки, но оно должно учитывать все ограничения задачи.

21 Как устроить автоматическое решение

Сначала сформулируем алгоритм обычными словами. Выбираем возможный порядок букв над столбцами. Для каждой строки ищем все полные наборы, которые сохраняют известные цифры и дают нужное F. Затем пробуем выбрать по одному набору для каждой строки так, чтобы выбранные наборы были различны. Если это возможно, выбранный порядок подходит.

Здесь есть два уровня выбора. Первый — порядок столбцов, общий для всей таблицы. Второй — заполнение каждой конкретной строки. Условия относятся сразу к обоим уровням, поэтому нельзя менять порядок столбцов отдельно для каждой строки.

Перед кодом разберём используемые конструкции. Список хранит несколько элементов в заданном порядке: rows = [(0, 1), (1, 0)]. Индексы начинаются с нуля, поэтому rows[0] — первый элемент. Метод append добавляет элемент в конец списка. len(rows) возвращает число элементов.

Словарь связывает имя и значение. Например, {'z': 0, 'x': 1, 'y': 1} означает z = 0, x = 1, y = 1. Обращение values['x'] достаёт значение, записанное для x.

Чтобы совместить подписи и цифры, используется zip. Для order = ('z', 'x', 'y') и row = (0, 1, 1) вызов zip(order, row) связывает z с 0, x с 1, y с 1. Вызов dict(zip(order, row)) превращает эти пары в словарь. Обе последовательности у нас одинаковой длины.

Функцию в Python объявляют словом def. Внутри её блока пишут вычисление, а return возвращает результат туда, где функцию вызвали. В нашей программе f(values) получит словарь значений, достанет из него переменные и вычислит формулу.

Для пустой клетки используем None — специальное значение «здесь пока нет заданной цифры». Проверка a is None определяет именно такой пропуск. Нельзя проверять пропуск условием not a: известный ноль тоже считается ложным, и программа ошибочно примет его за пустую клетку.

Множество, создаваемое set(...), хранит различные элементы без повторений. Если было три кортежа и два из них совпадали, после преобразования в множество останутся два элемента. Поэтому len(set(restored)) == len(restored) означает, что все восстановленные строки различны.

Слово break завершает ближайший цикл, внутри которого оно стоит. Например, обнаружив несовпадение одной известной клетки, можно прекратить проверять оставшиеся клетки этого набора. Переменная fits будет хранить True или False: подходит ли набор по известным цифрам.

В product(*candidates) звёздочка раскрывает список списков в отдельные аргументы. Если candidates = [L1, L2, L3], вызов равен product(L1, L2, L3). Он перебирает все способы взять один элемент из L1, один из L2 и один из L3. Здесь выбираются целые строки, а не отдельные нули и единицы.

Наконец, ''.join(order) соединяет буквы порядка в одну строку без разделителей. Множество answers убирает повторную запись одного ответа, если ему подходят разные заполнения. Метод answers.add(...) добавляет ответ, а sorted(answers) возвращает ответы в упорядоченном списке.

22 Полная программа с учётом всех строк

Ниже программа для примера с четырьмя переменными из раздела 15. Она разделена на три последовательные части для удобства чтения. При запуске помести их в один файл в указанном порядке. В полном коде нет внешних зависимостей, нужны только инструменты стандартной библиотеки.

Сначала задаём формулу и фрагмент. В каждой записи fragment первый элемент — строка входных значений, второй — требуемое F. Пропуски записываем как None.

```
from itertools import product, permutations

def f(values):
    x = values['x']
    y = values['y']
    z = values['z']
    w = values['w']
    return int(((not x) or y) and
               ((not y) or z) and
               ((not z) or w))

names = 'xyzw'
fragment = [
    ((0, None, 1, None), 1),
    ((1, 0, None, 0), 1),
    ((None, 0, 1, 1), 1),
]

all_rows = list(product((0, 1), repeat=len(names)))
answers = set()
```

Имя names содержит все переменные по одному разу. Его порядок задаёт только исходный список для перестановок. Каждая строка all_rows пока означает значения столбцов C1, C2, C3, C4. Смысл этих столбцов появится после выбора order.

Затем для каждого порядка столбцов собираем варианты каждой строки.

```
for order in permutations(names):
    candidates = []

    for pattern, wanted in fragment:
        matching = []

        for row in all_rows:
            fits = True

            for col in range(len(names)):
                if (pattern[col] is not None and
                        pattern[col] != row[col]):
                    fits = False
                    break

            values = dict(zip(order, row))
            if fits and f(values) == wanted:
                matching.append(row)

        candidates.append(matching)
```

Вызов range(len(names)) создаёт номера позиций: для четырёх переменных это 0, 1, 2, 3. Переменная pattern хранит строку с пропусками, row — проверяемый полный набор, wanted — нужный результат функции именно для этой строки.

В условии с and сначала проверяется, что клетка известна, затем сравнивается её значение с кандидатом. Несовпадение известной клетки исключает набор. Пропуск ничего не запрещает. Если цифры согласованы, словарь values подставляет их под выбранные имена, а f(values) проверяет формулу.

Список matching содержит все варианты одной строки. Список candidates собирает такие списки для всех строк. Если хотя бы для одной строки вариантов нет, невозможно выбрать по одному из каждого списка, и следующий перебор не даст ни одного восстановления.

Следующая часть продолжает внешний цикл for order. Поэтому for restored начинается с отступа в четыре пробела, а вложенные в него строки имеют дополнительные отступы, как показано в коде. Последний print записан без отступа: он выполняется после завершения всего перебора.

```
    for restored in product(*candidates):
        if len(set(restored)) == len(restored):
            answers.add(''.join(order))
            break

print(sorted(answers))
```

Результат программы: ['zxwy']. Квадратные скобки и кавычки — оформление списка в Python. В поле ответа задачи следует записать только zxwy.

Почему можно сделать break после первого успешного восстановления? Мы уже доказали, что данный порядок столбцов допустим. Другие заполнения для него не добавят нового порядка. Но внешний цикл продолжится: необходимо проверить остальные перестановки и убедиться, что нет другого ответа.

Почему алгоритм ничего не пропускает? Все возможные порядки перебирает permutations. Все бинарные строки перебирает product. Проверка клетки исключает ровно несовпадения с известными данными, проверка f — неправильное значение

функции. Последний product рассматривает все совместные варианты заполнения, а set отсеивает повторения. Значит, любой допустимый порядок будет найден.

Почему алгоритм не принимает лишнего? Ответ добавляется только после выбора полного набора различных строк, каждая из которых уже проверена и по известным клеткам, и по формуле. Это непосредственно соответствует условию задачи.

23 Как менять программу под другую задачу

Для обычной задачи изменяются три вещи: вычисление внутри f, строка names и содержимое fragment. В каждом шаблоне должно быть ровно столько входных клеток, сколько букв в names. Значение F указывается отдельно и не входит в этот счёт.

Для примера из раздела 14 замени функцию и входные данные следующими. Остальная часть программы останется той же.

```
def f(values):
    x = values['x']
    y = values['y']
    z = values['z']
    return int((x or y) and ((not x) or z))

names = 'xyz'
fragment = [
    ((None, None, 0), 1),
    ((None, None, 0), 1),
    ((0, 1, 1), 1),
]
```

Ожидаемый ответ: ['yzx']. Одинаковые шаблоны первых двух строк нужно сохранить как две записи. Если заранее удалить один из них как «дубликат», исчезнет часть условия: после восстановления строки должны быть разными.

Для примера с разными F из раздела 16 та же функция и names сохраняются, меняется только fragment:

```
fragment = [
    ((None, 0, 0), 0),
    ((0, None, 1), 1),
    ((None, 1, 1), 0),
]
```

Ожидаемый ответ: ['zxy']. Программа уже поддерживает разные значения F, потому что сравнивает результат с wanted для каждой строки отдельно.

Если в нестандартном упражнении значение F тоже скрыто и не задано словами, можно обозначить его None. Тогда проверку if fits and f(values) == wanted: нужно заменить на if fits and (wanted is None or f(values) == wanted):. Условие о различии входных наборов сохраняется.

Если явно перечислены все строки с F = 0 или все с F = 1, дополнительно проверь полноту. Посчитай по полной таблице, сколько всего наборов дают указанное F, и сравни с числом строк фрагмента. Когда числа совпадают и все восстановленные строки различны и имеют нужное F, полнота обеспечена: ни один подходящий набор уже не может остаться за пределами фрагмента.

Программа предназначена для небольших фрагментов задания 2. Число вариантов совместного заполнения растёт, когда строк много и в них почти нет известных клеток. Для трёх или четырёх переменных и небольшого числа строк это понятный прямой перебор; для произвольно больших таблиц потребовались бы другие способы поиска.

Если получен пустой список [], проверь скобки и направление стрелок, переписанные цифры, размер строк, значения F и имена переменных. Если получено несколько строк-ответов, проверь, не пропущено ли дополнительное условие. Две распечатки одного порядка из другого варианта кода могут означать всего лишь два заполнения; в нашей программе множество answers убирает такие повторы.

24 Самые частые ошибки и их причины

Ошибка	Почему неверно	Как проверить себя
Из A ∨ B = 1 делают A = B = 1	Достаточно одной единицы	Подставить A = 1, B = 0
Из A ∧ B = 0 делают A = B = 0	Достаточно одного нуля	Подставить A = 0, B = 1
Считают 0 → 0 ложным	Нарушение стрелки только 1 → 0	Раскрыть ¬A ∨ B
Путают A → B с B → A	У стрелки есть направление	Проверить набор 0,1
Считают 0 ≡ 0 ложным	Эквивалентность проверяет совпадение	Сравнить два нуля
Принимают пропуск за 0	Пропуск допускает оба значения	Перечислить оба заполнения
Переставляют буквы отдельно в строках	Подписи общие для столбцов	Подписать столбцы один раз
Считают фрагмент полной выборкой	Часть подходящих строк могла не попасть в условие	Найти слово «все» и его точный смысл
Используют один набор дважды	Строки должны различаться	Сравнить восстановленные входы
Сопоставляют строки по номеру	Порядок вывода программы произволен для задачи	Сравнивать цифры, а не позиции
Пишут x <= y == z	Python выполняет цепочку сравнений	Явно поставить (x <= y) == z
Выдают порядок номеров вместо букв	Ответ читается слева направо по столбцам	Подписать C1, C2, C3, C4

Ещё одна ошибка менее заметна: рассуждать по «похожести» столбцов. Например, если две переменные иногда равны, это не значит, что они всегда должны иметь одинаковые значения. Используй только доказанное ограничение — во всех рассматриваемых наборах или в конкретной строке.

Полезная проверка законченного решения состоит из четырёх действий. Сохранены ли все известные цифры? Одинаковы ли подписи столбцов во всех строках? Даёт ли исходная формула требуемое F в каждой строке? Различны ли полностью восстановленные входные наборы? Если условие требует полноту, отдельно проверь и её.

25 Упражнения от отдельных операций к полной задаче

Решай упражнения письменно. Сначала запиши объяснение, затем проверяй ответ. Для упражнений про неизвестные столбцы символ * означает пропуск, а показанные строки считаются различными. Все примеры ниже учебные и составлены для этого конспекта.

Упражнение 1. При A = 0, B = 1 найди ¬A, A ∧ B, A ∨ B, A → B, B → A, A ≡ B и A ⊕ B. Для каждой стрелки назови её левую и правую части.

Упражнение 2. Перечисли все пары A, B, при которых: а) A ∧ B = 0; б) A ∨ B = 1; в) A → B = 0; г) A ≡ B = 1. Не сокращай ответ до одного примера, если возможностей несколько.

Упражнение 3. При x = 0, y = 0, z = 1 вычисли F = ((¬x ∨ y) → z) ∧ (x ≡ y). Выпиши промежуточные значения. Затем измени только z на 0 и вычисли F снова.

Упражнение 4. Построй полную таблицу F = ¬(x ∧ y) и полную таблицу G = ¬x ∧ ¬y. Найди строки, на которых результаты отличаются. Какое правило преобразования нельзя использовать?

Упражнение 5. Без построения всех восьми строк найди наборы xyz, на которых F = (x ∨ y) ∧ (x → z) равна 0. Объясни случаи отдельно.

Упражнение 6. Рассматривается F = x → y. Дан фрагмент из одной строки: C1 = 0, C2 = 1, F = 0. Определи подписи столбцов. Почему ответ не xy?

Упражнение 7. У функции F = (x ∨ y) ∧ (x → z) показаны все строки с F = 1. В неизвестном столбце ровно две единицы. Какая это переменная? Почему тот же подсчёт нельзя без изменений применить к произвольному фрагменту из трёх строк?

Упражнение 8. Формула F = (a ∨ b) ∧ (a → c). Найди порядок столбцов.

Строка	C1	C2	C3	F
1	*	0	*	1
2	*	0	*	1
3	1	1	0	1

Упражнение 9. Для некоторого порядка столбцов у первой строки есть ровно один допустимый полный набор 101, у второй тоже только 101, у третьей только 011. Каждая строка отдельно согласуется с формулой. Допустим ли порядок для фрагмента из трёх различных строк?

Упражнение 10. Известно, что C1 = y, C2 = w, C3 = x, C4 = z. Запиши ответ. Затем для строки столбцов 1, 0, 1, 0 выпиши значения в порядке x, y, z, w. Почему это два разных порядка записи?

Упражнение 11. Требуется вычислить (x → y) ≡ z. Ученик написал x <= y == z. Найди результат ошибочной и правильной записи при x = y = z = 0. Объясни расхождение.

Упражнение 12. Формула F = ¬((x → z) ∧ (z → w) ∧ (w → y)). Найди порядок столбцов. Внимательно учти внешнее отрицание.

Строка	C1	C2	C3	C4	F
1	0	*	1	*	0
2	1	0	*	0	0
3	*	0	1	1	0

Упражнение 13. Формула F = x ≡ y. Может ли фрагмент её таблицы без дополнительных сведений о подписях однозначно различить x и y? Объясни ответ, даже если известны все клетки.

Упражнение 14. Проверь утверждения: а) A → B равносильно B → A; б) A → B равносильно ¬B → ¬A; в) ¬(A ∨ B) равносильно ¬A ∨ ¬B. Для неверных утверждений приведи конкретные значения, на которых результаты различаются.

Упражнение 15. Возьми программу из раздела 22 и подставь данные из раздела 16. Сначала предскажи результат вручную. Затем замени строку if len(set(restored)) == len(restored): на if True:, сохранив отступы, и повтори эксперимент на данных раздела 14. Теперь проверка различия строк всегда пропускает выбранное заполнение. Какие лишние порядки появятся и почему?

Упражнение 16. Для четырёх переменных сколько существует: а) наборов входных значений; б) порядков подписей столбцов; в) способов независимо заполнить пять пропусков, до проверки формулы? Объясни, почему ответы не обязаны совпадать.

26 Ответы с объяснениями

1. Последовательно: 1, 0, 1, 1, 0, 0, 1. В A → B слева 0, справа 1, запрещённой пары нет. В B → A слева 1, справа 0, поэтому результат 0. Значения A и B различаются: эквивалентность ложна, исключающее ИЛИ истинно.

2. а) 00, 01, 10: хотя бы один ноль. б) 01, 10, 11: хотя бы одна единица. в) Только 10: условие выполнено, требование нет. г) 00 и 11: значения совпадают. Порядок в каждой паре — сначала A, затем B.

3. При x = y = 0 получаем ¬x = 1, ¬x ∨ y = 1, x ≡ y = 1. Поэтому F = (1 → z) ∧ 1 = z. При z = 1 результат 1, при z = 0 результат 0. Мы не пересчитывали всю формулу повторно, потому что изменился только z.

4. В порядке xy = 00, 01, 10, 11 значения F: 1, 1, 1, 0; значения G: 1, 0, 0, 0. Различия на 01 и 10. Нельзя переносить отрицание внутрь И, сохраняя И между отрицаниями. Правильно: ¬(x ∧ y) = ¬x ∨ ¬y.

5. Первая часть x ∨ y ложна при x = y = 0, что даёт 000 и 001. Стрелка x → z ложна при x = 1, z = 0, что даёт 100 и 110. Итого 000, 001, 100, 110. Совпадений между этими двумя группами нет.

6. Стрелка ложна только при x = 1, y = 0. В первом столбце 0, значит, там y. Во втором 1, значит, там x. Ответ yx. При подписях xy строка означала бы x = 0, y = 1 и давала бы F = 1, что противоречит условию.

7. Все единичные наборы xyz: 010, 011, 101, 111. У x две единицы, у y три, у z три. Следовательно, столбец с двумя единицами — x. В произвольном фрагменте один набор может быть опущен, и число единиц для y или z тоже может стать равным двум. Полнота здесь была существенна.

8. Ответ cab. Подходящие наборы в порядке abc: 010, 011, 101, 111. Только a равна 0 в двух различных подходящих наборах, поэтому C2 = a. В третьей строке a = 1, значит, c = 1. Из оставшихся столбцов C1 равен 1, C3 равен 0: C1 = c, C3 = b. Первые две строки восстанавливаются как 001 и 101 в порядке cab, в любом из двух порядков.

9. Порядок недопустим. Первую и вторую строки пришлось бы восстановить одинаково. Возможность подобрать строку по отдельности не гарантирует возможности подобрать разные строки для всего фрагмента.

10. Ответ ywxz. Данная строка сообщает y = 1, w = 0, x = 1, z = 0. В порядке xyzw получаем 1100. Строка ответа описывает подписи над столбцами, а набор 1100 — значения переменных в выбранном привычном порядке.

11. Правильная запись (x <= y) == z даёт (0 <= 0) == 0, затем True == 0, результат False. Ошибочная запись даёт (0 <= 0) and (0 == 0), то есть True. Скобки определяют, сравнивается ли результат стрелки с z или проверяются два отдельных сравнения.

12. Ответ wxyz. Из F = 0 и внешнего отрицания следует, что внутреннее И равно 1. Все три стрелки истинны, поэтому значения в цепочке x, z, w, y сначала могут быть нулями, затем единицами. Из клеток следует порядок позиций C2, C4, C1, C3 вдоль этой цепочки. Значит, C2 = x, C4 = z, C1 = w, C3 = y. В порядке столбцов это wxyz.

13. Не может. Обмен x и y не меняет значение $x \equiv y$ при любом наборе. Если один порядок подходит, обменённый тоже подойдёт. Нужна дополнительная информация, нарушающая эту неразличимость, или другой вопрос задачи.

14. а) Неверно: при A = 0, B = 1 первое выражение равно 1, второе 0. б) Верно: раскрытие второй стрелки даёт $B \vee \neg A$, то же значение, что $\neg A \vee B$. в) Неверно: при A = 1, B = 0 слева $\neg 1 = 0$, справа $0 \vee 1 = 1$. Правильно менять ИЛИ на И: $\neg A \wedge \neg B$.

15. Для раздела 16 ответ zxy. Для раздела 14 без проверки различия появляются порядки xyz, xzy, yxz вместе с правильным yzx. В лишних вариантах обе первые строки используют один и тот же единственный подходящий набор с нулём в назначенной переменной. Они проходят построчную проверку, но нарушают условие о разных строках.

16. а) $2^4 = 16$: у каждой из четырёх переменных два независимых значения. б) $4! = 24$: буквы переставляются без повторений. в) $2^5 = 32$: у каждой из пяти пустых клеток сначала два выбора. После проверки формулы и остальных условий часть заполнений может быть исключена.

27 Как закрепить материал

Первый этап — восстановить таблицы операций по их смыслу. Особенно важно уметь словами объяснить, почему $0 \to 0 = 1$, почему $1 \vee 1 = 1$ и почему $0 \equiv 0 = 1$. Если эти случаи понятны, самые частые ошибки уже предотвращены.

На втором этапе вычисляй составные формулы через промежуточные части. Не пытайся сразу считать длинную строку целиком. Проверяй себя на двух наборах, отличающихся одной переменной, и объясняй, почему результат изменился или остался прежним.

На третьем этапе решай обратную задачу: по $F = 0$ или $F = 1$ выводи ограничения. Каждый раз спрашивай себя, требуется ли совместное выполнение условий или достаточно одного из нескольких случаев. Выписывай все ветви, если возникло «или».

На четвёртом этапе переходи к фрагментам с неизвестными столбцами. Для каждой найденной подписи записывай причину: постоянное значение, невозможное число нулей, равенство переменных, запрет конкретного набора или неповторение строк. Причина должна оставаться проверяемой без догадки «похоже, это x».

Затем используй Python как независимую проверку. Сначала сравни напечатанные подходящие наборы со своим ручным списком, затем проверь порядок столбцов. Если результаты расходятся, найди первую конкретную строку, где ручное вычисление и код дают разные F, и пересчитай её по частям.

После каждого решённого задания полезно ответить на два вопроса: «Какое условие определило самый первый столбец?» и «Почему другой оставшийся порядок не подходит?». Именно эти ответы превращают выполненное упражнение в переносимый способ рассуждения.

Источники и границы материала

1. РЕШУ ЕГЭ. Подборка «Построение таблиц истинности логических выражений. Строки с пропущенными значениями». Использована для проверки формата условий и различия между фрагментом и всеми наборами. https://inf-ege.sdamgia.ru/test?theme=352

2. Официальная документация Python. Раздел Expressions, подразделы Comparisons и Boolean operations. Использована для уточнения поведения and, or, not, сравнений и цепочек сравнений. https://docs.python.org/3/reference/expressions.html

3. Официальная документация Python. Модуль itertools, функции product и permutations. https://docs.python.org/3/library/itertools.html

Материал посвящён двузначной логике и задачам на восстановление столбцов таблиц истинности. Разборы и упражнения составлены для объяснения методов; они не являются утверждением о составе конкретного экзаменационного варианта. Формулы и ответы в учебных примерах проверены полным перебором.